



Software Development **THE SHOP**



Overview

The Shop – Abstract

Architecture – Diagram & Connection

Design – CI/CD

Cloud – Azure Resource Group

Progress – Testing & Monitoring

SCRUM – Product Backlog

Feedback – Future Improvements

Conclusion – Challenges & Learnings



The Shop

The Shop is a cloud-native backend for a mini online marketplace that manages products, customers, and orders via a RESTful JSON API. It is deployed on Azure App Service with Azure SQL Database for persistence, CI/CD pipelines for automated build-test-deploy, and Application Insights for telemetry.

Our Goal



Specific

Build an e-commerce API and implement a full DevOps pipeline that automates testing, deployment, and monitoring.

Measurable

Ensure 100% automated deployments, maintain at least 80% test coverage, and achieve stable API uptime above 99%.

Attainable

Use established DevOps tools like Docker, Azure, and CI/CD pipelines, dividing responsibilities across the team to deliver each component.

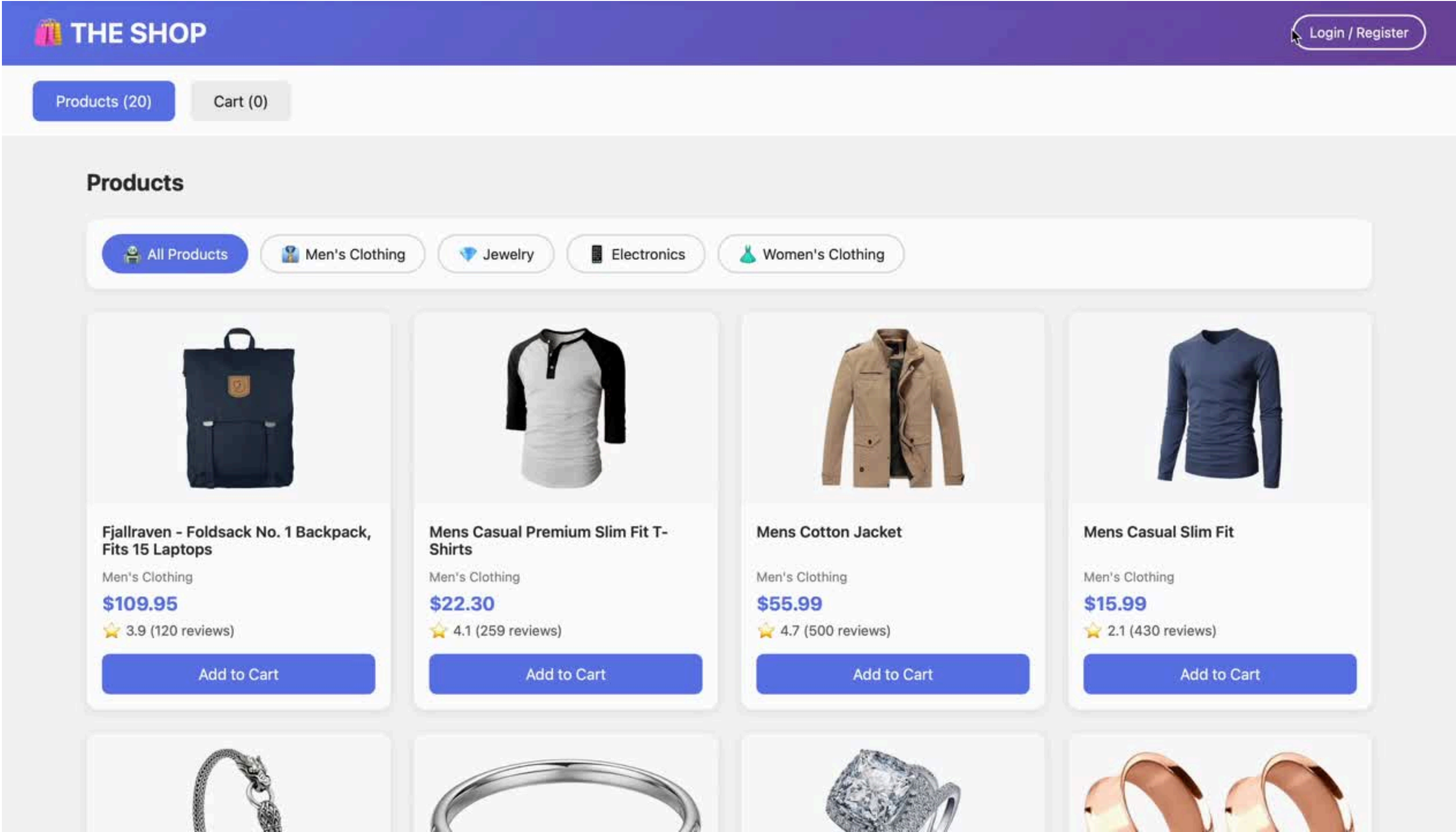
Relevant

This goal directly supports the need to streamline communication between shops and backend systems through a reliable, scalable API.

Timebound

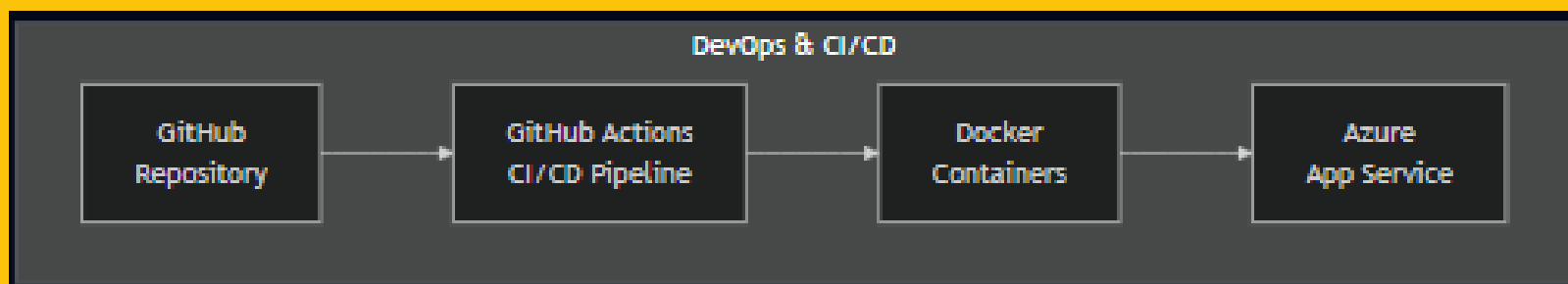
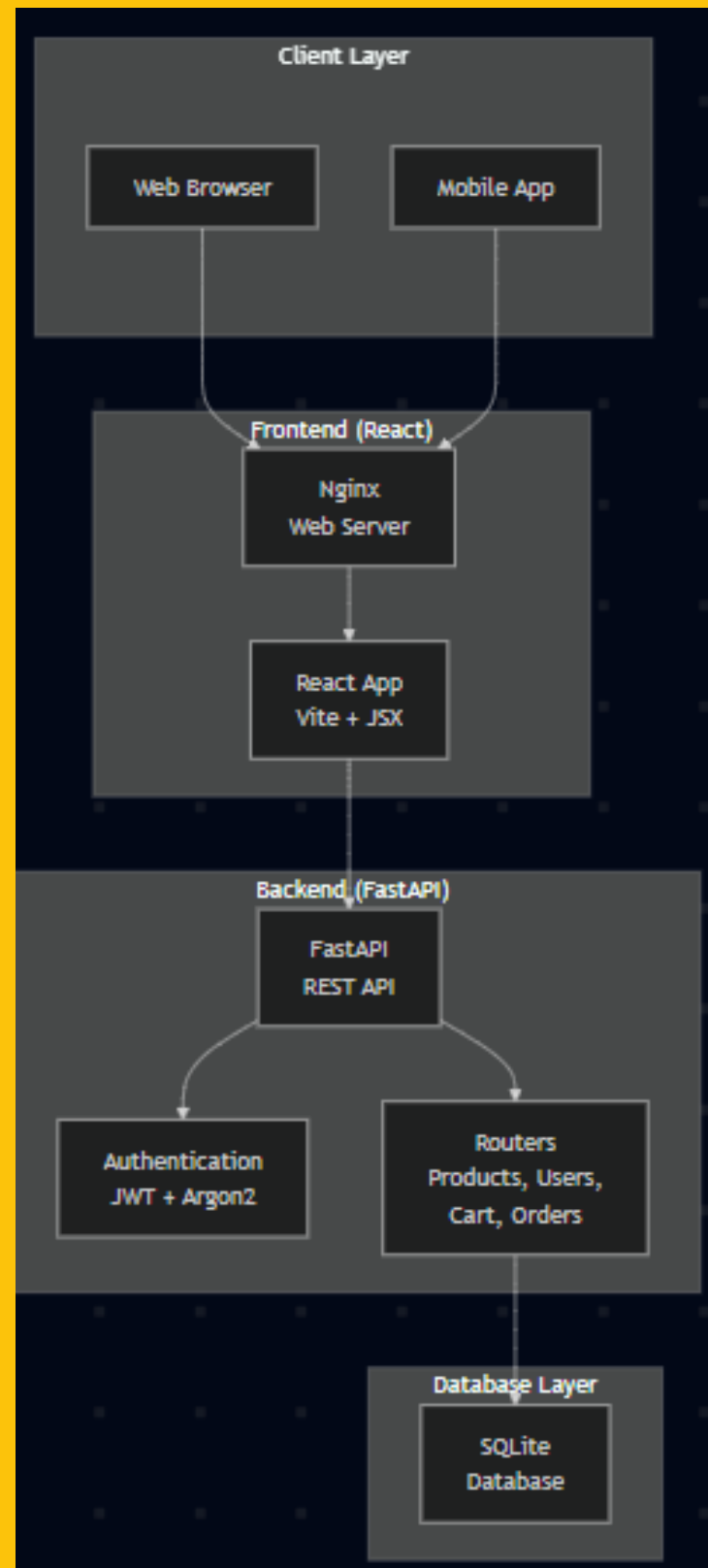
Complete all development, testing, deployment, and monitoring steps within the project's defined deadline.

OVERVIEW



Architecture

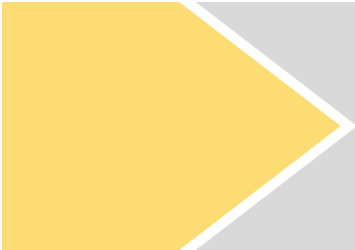
- **API layer:** A Python backend with organized routers for /products, /customers, and /orders, all using a shared database.py module to manage connections.
- **Data layer:** An Azure SQL Database storing normalized tables for products, customers, and orders, with foreign keys linking related records.
- **Integration:** The API uses a single secure connection string and can generate SAS URLs for product images stored in Azure Blob Storage.



SOLID PRINCIPLES



Single Repository



Open/Closed



Liskov Substitution



Interface Segregation



Dependency Inversion

SCRUM

S0

Preparation (Setup & Foundations)

- Defined team roles
- Organized backlog
- Set up architecture & cloud environment

S1

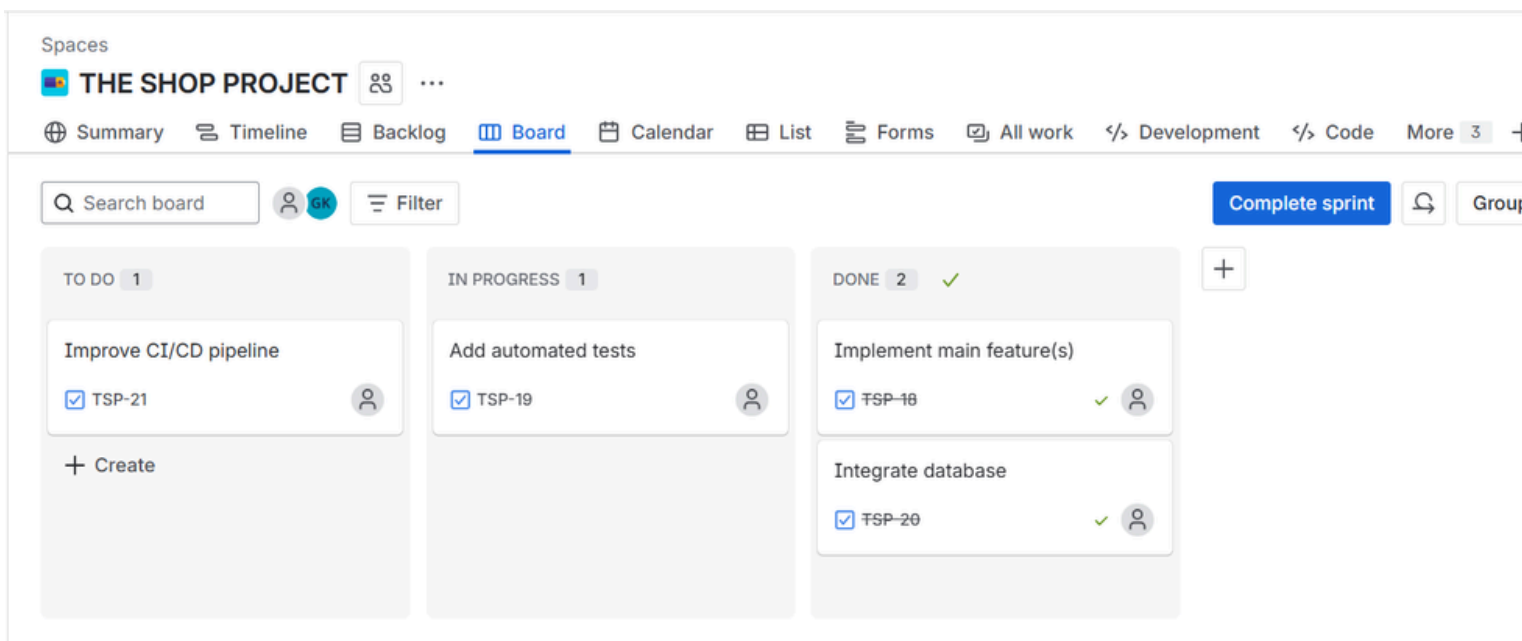
Create skeleton application and backend foundation

- Built initial app structure
- Deployed first version on Azure
- Established CI/CD pipeline

S2

API development and Frontend

- Delivered core API + frontend features
- Added automated tests
- Strengthened CI/CD with quality checks



SCRUM



S3

Testing CI pipeline

- Expanded app functionality & user experience
- Improved interface design
- Enhanced testing coverage for stability & usability

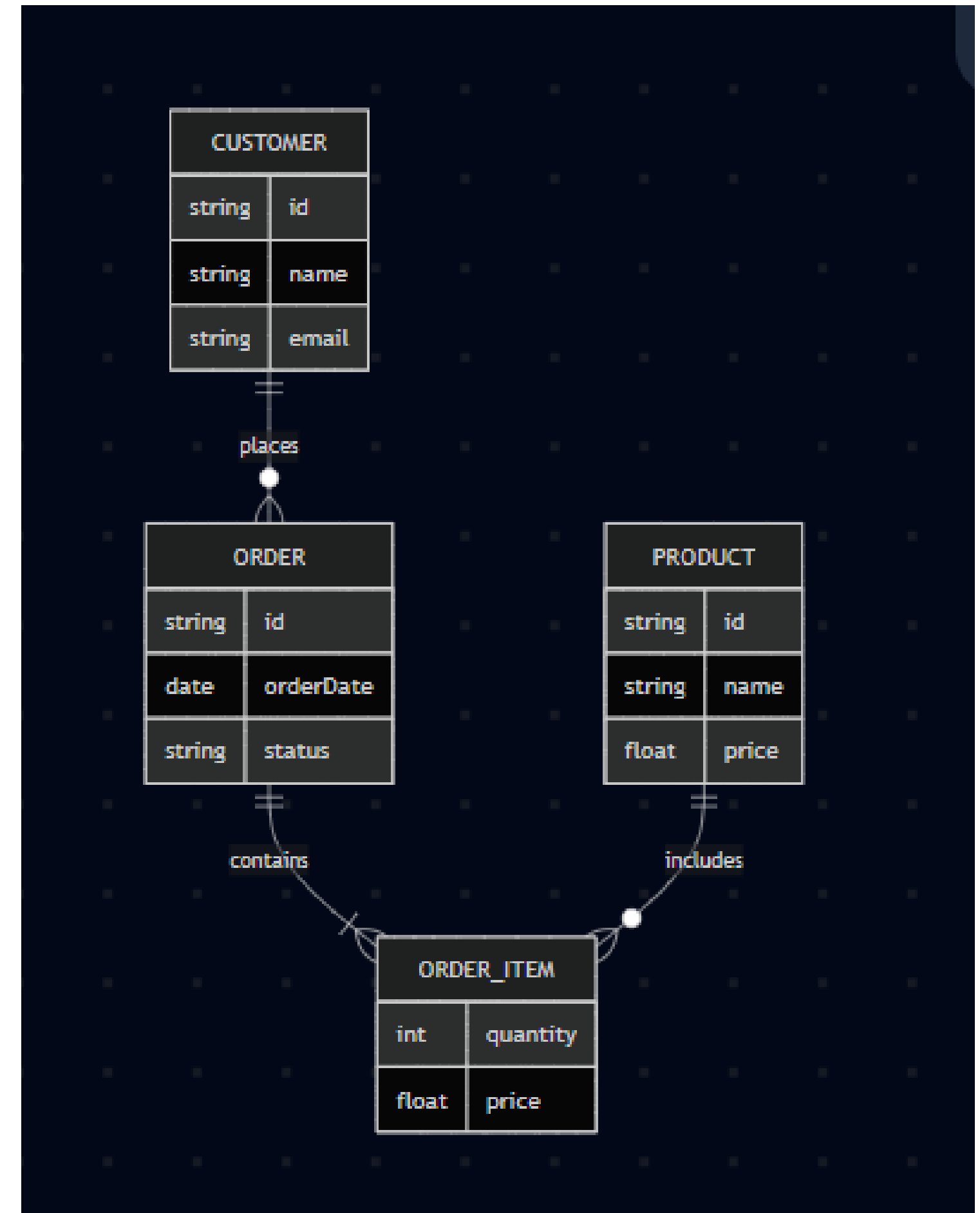
S4

CD pipeline and deployment to Azure(in progress)

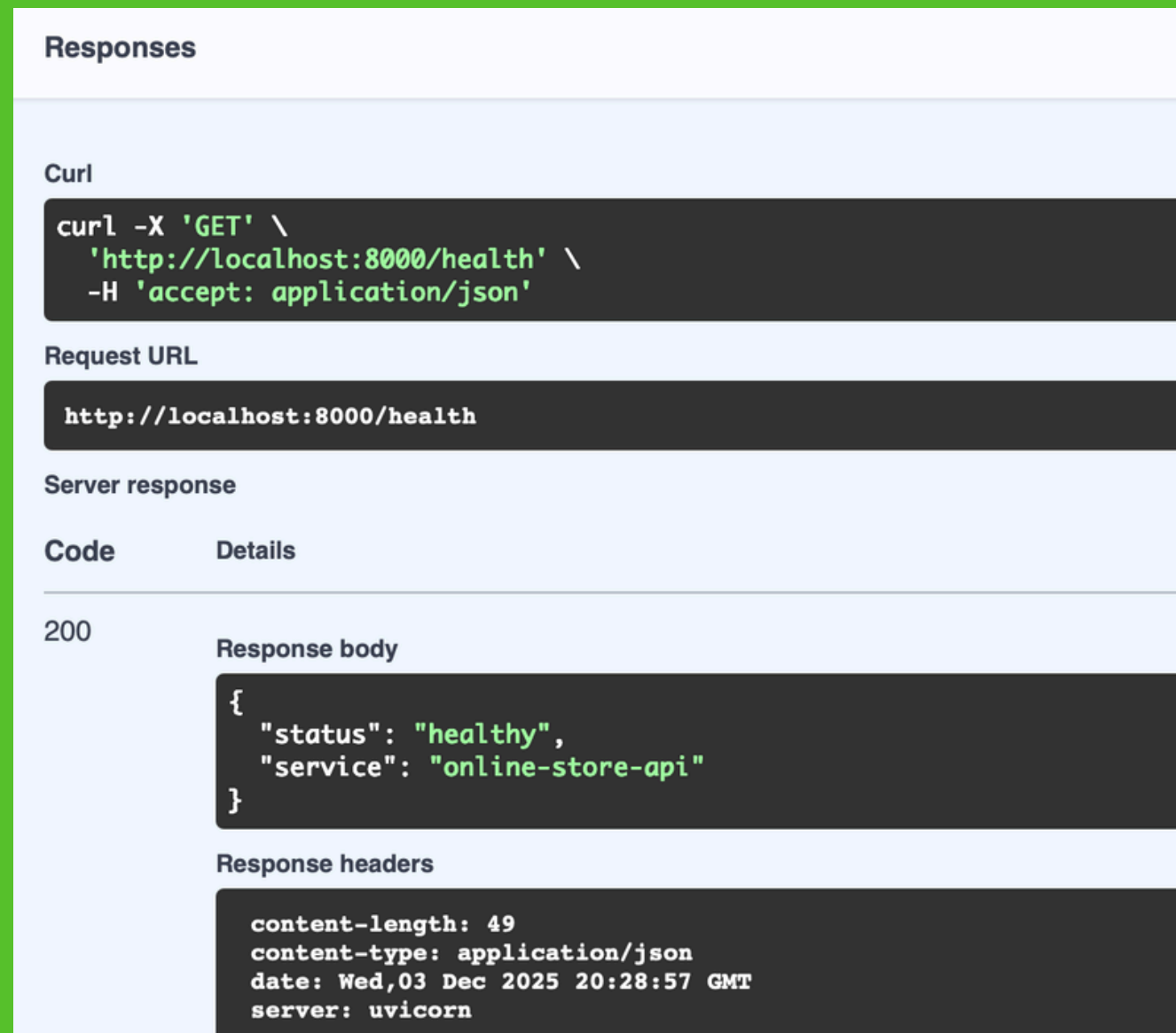
- Fine-tuned CI/CD pipeline for performance
- Conducted load testing & bug fixes
- Preparing Azure deployment for production release
- blob storage
- monitoring

Focus on building a solid foundation,
delivering incremental value, and
ensuring reliability at each step

ER-diagram



DOCKER AND HEALTH



The screenshot displays a web browser interface for a health check. It shows the curl command used to request the health status, the request URL, and the resulting JSON response body and headers.

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8000/health' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8000/health
```

Server response

Code	Details
200	Response body <pre>{ "status": "healthy", "service": "online-store-api" }</pre> Response headers <pre>content-length: 49 content-type: application/json date: Wed, 03 Dec 2025 20:28:57 GMT server: uvicorn</pre>

Setup:

- Backend → Python (FastAPI)
- Frontend → React (Nginx)
- docker-compose.yml → runs both together

Benefits:

- Consistent environments (dev, test, prod)
- Easy cloud deployment (Azure, AWS)
- Isolated dependencies
- Fast startup & scaling

Health Checks:

- Automated tests for availability & responsiveness
- Endpoint: /docs (FastAPI auto-docs)
- Method: HTTP GET → Success = 200
- Retry: 5 attempts, 15s intervals

tests

===== tests coverage =====				
coverage: platform darwin, python 3.9.6-final-0				
Name	Stmts	Miss	Cover	Missing
Backend/__init__.py	0	0	100%	
Backend/auth.py	47	9	81%	60, 67, 69, 79-84
Backend/crud.py	108	11	90%	65-69, 171-176
Backend/database.py	14	1	93%	13
Backend/main.py	21	0	100%	
Backend/models.py	52	0	100%	
Backend/routers/__init__.py	0	0	100%	
Backend/routers/auth.py	43	4	91%	60, 77, 84, 91
Backend/routers/cart.py	32	3	91%	37, 52, 64
Backend/routers/orders.py	44	16	64%	34, 36, 48, 64-78, 88-95
Backend/routers/products.py	33	2	94%	50, 58
Backend/routers/users.py	27	1	96%	23
Backend/schemas.py	76	0	100%	
Backend/seed.py	45	45	0%	2-80
TOTAL	542	92	83%	
===== 157 passed in 3.09s =====				

Unit Tests (157 total passed)

- Test individual components in isolation (models, schemas, CRUD, auth, DB sessions)
- Example: User creation logic
- Tools: pytest, pytest-asyncio, pytest-cov

Integration Tests

- End-to-end API validation with test DB + HTTP client
- Covers login flow, protected routes, database operations, error handling
- Tools: httpx, FastAPI TestClient

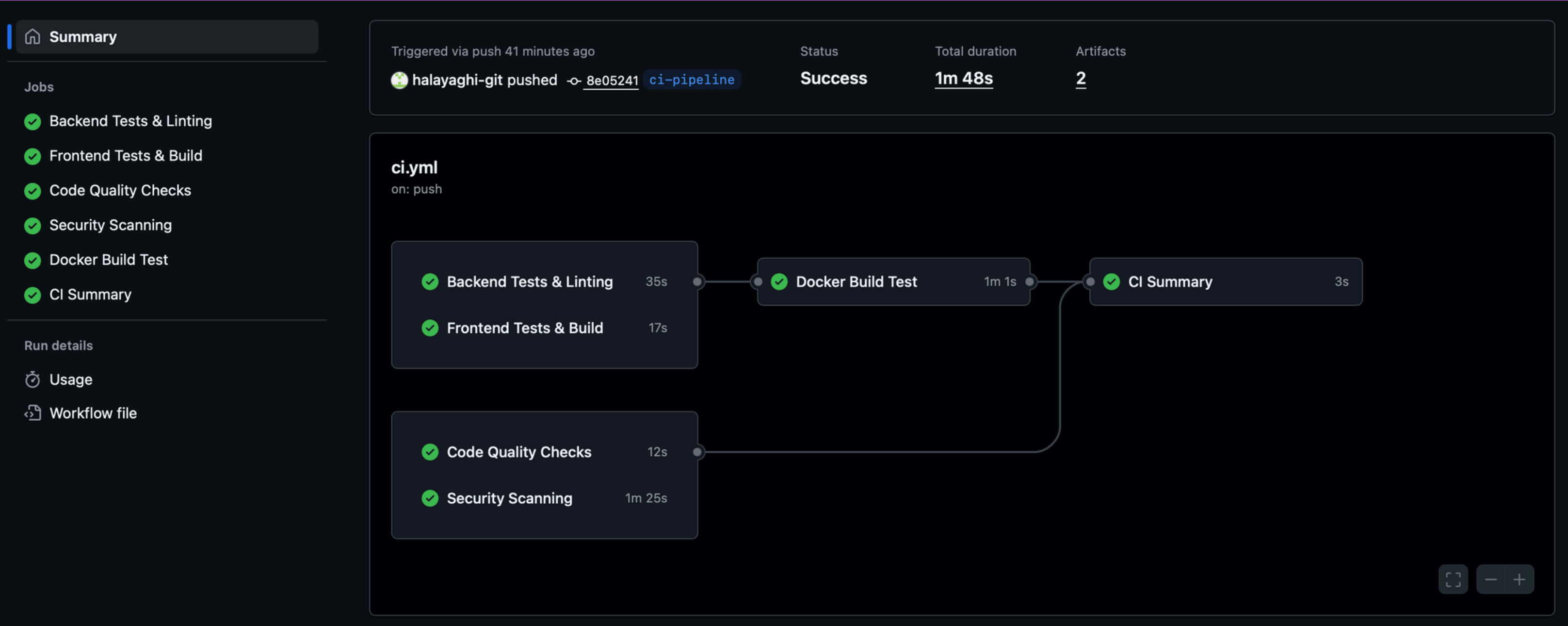
Security Tests

- Static scans outside CI pipeline
- Tools: bandit, pip-audit, Trivy
- Checks: SQL injection, secrets, CVEs, unsafe file ops
- Best Practices: Argon2 hashing, JWT auth, Pydantic validation, env-based secrets

Test Results Summary

- 157 tests passed successfully
- 83% backend coverage
- All critical paths tested and verified

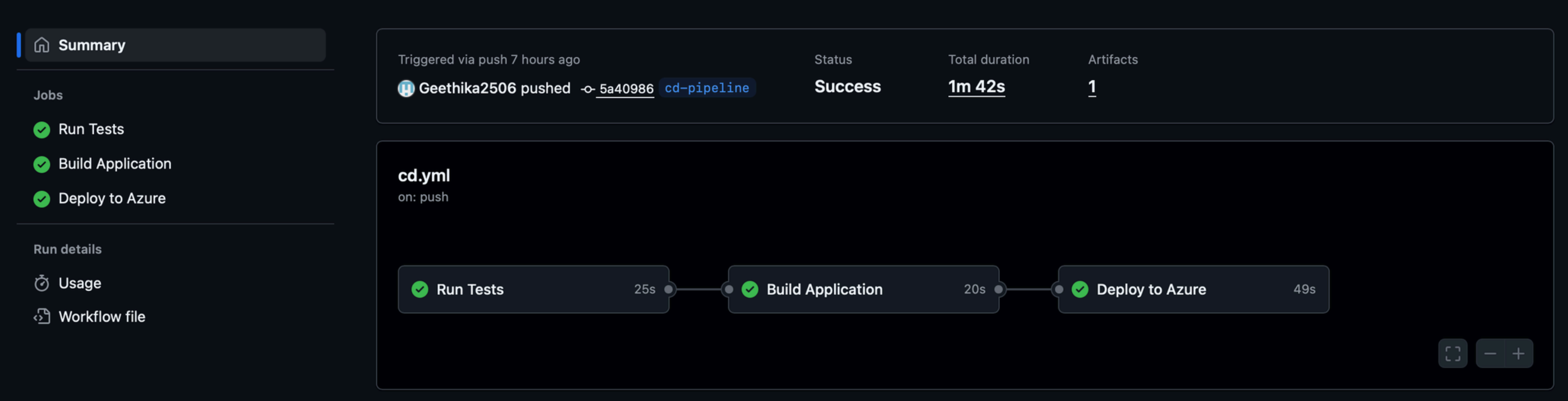
CI pipeline



CI pipeline

Job	Purpose	Key Tools
1. Frontend Test & Build	Validates Python code quality and runs unit tests	flake8 (linting), pytest (testing), pytest-cov (coverage)
2. Frontend Test & Build	Validates React code and builds production bundle	eslint (linting), npm (package manager), vite (build tool)
3. Docker Build Test	Ensures Docker images build correctly and compose works	docker, docker-compose, docker/build-push-action
4. Code Quality	Scans for security vulnerabilities	trivy (vulnerability scanner), CodeQL
5. Security Scan	Deep security analysis of code and dependencies	bandit (Python SAST), pip-audit, safety, gitleaks (secrets), npm audit
6. CI Summary	Reports final status of all jobs	GitHub Actions summary

CD pipeline



CD pipeline

Job	Purpose	Tools/Libraries
1. Test	Run unit tests to verify code works	pytest, pytest-asyncio, httpx
2. Build	Package application into deployable zip	pip, zip
3. Deploy	Deploy to Azure App Service	Azure CLI, azure/login, azure/webapps-deploy

Cloud



Our cloud setup runs the application on Azure App Service using containerized backend and frontend images stored in Azure Container Registry. The backend includes a database layer that works with SQLite locally and is ready for Azure SQL in production. Application Insights gives us real-time monitoring, helping us understand performance and user activity.

Key resources

- App Service running our backend and frontend in containers
- Container Registry storing our Docker images
- Application Insights monitoring performance and logs
- Database layer ready for Azure SQL when moving to full production

Monitoring and reliability

- Application Insights tracks requests, errors, and performance
- Real-time logs and dashboards help us detect issues quickly
- Shows slow endpoints, failures, and user activity patterns
- Helps maintain reliability as new versions are deployed

Cloud Benefits

- Smooth deployment using containerized apps
- Scales automatically as traffic grows
- All resources organized inside one Azure Resource Group

Deployment Integration

- App Service pulls the latest images from Container Registry
- Application Insights updates automatically with each deployment

Cloud



← CD Pipeline

✔ feat: use Azure Container Registry theshopproject #13 Re-run all jobs ...

Summary

Jobs

- ✔ Run Tests
- ✔ Build and Push to ACR
- ✔ Deploy to Azure

Run details

- Usage
- Workflow file

Triggered via push 12 minutes ago

Geethika2506 pushed -o- 5391fb1 [cd-pipeline](#) **Success** **1m 56s** **Artifacts** -

cd.yml
on: push

✔ Run Tests 22s → ✔ Build and Push to ACR 1m 3s → ✔ Deploy to Azure 24s

Build and Push to ACR summary ...

Images Pushed to ACR

- Backend: `***.azurecr.io/theshop-backend:5391fb1eb7f2343369478d3903baa242188d0ab0`
- Frontend: `***.azurecr.io/theshop-frontend:5391fb1eb7f2343369478d3903baa242188d0ab0`

This image shows our deployment pipeline sending the backend and frontend containers to Azure Container Registry and updating the live application on Azure App Service. It connects directly to the cloud setup used in our project.

CHALLENGES & LEARNINGS

Repository & Branch Coordination

- We had many different branches and project structures at the start
- Merging was difficult and required constant pulling and pushing
- Next time we should start with a proper folder structure so everyone can follow the same layout

CI Pipeline Challenges

- CI workflows failed many times early on
- We needed to fix tests, Docker builds, and permissions across several runs
- Helped the team learn how to interpret pipeline errors

CD Pipeline Challenges

- Deploying to Azure App Service was challenging at first
- We had to research how to set up ACR, permissions, and connection settings
- Repeated failures helped us learn the full end-to-end deployment process

Scrum Tools

- We used Jira for sprint management, but it was not very user-friendly
- The team struggled at first to understand how it worked

Future Improvements

- Add more user features: wishlists, product reviews, and detailed order tracking
- Implement a secure payment method to streamline transactions
- Expand the product catalog with more items for variety and scalability
- Provide mobile app support with native applications and progressive web app capabilities

Team Roles

We didnt have rotating roles for the project



Geethika	Developer
Juliette	Developer
Hala	Developer
Nicolas	Product Manager
Andrea	Scrum master
Omar	Other Support



Feedback?



Madrid, Spain.
www.ie.edu